Dynamic Programming

0-1 Knapsack

0/1 Knapsack Problem

Given,

- W, total weight capacity
- List of the weights of some items, w_i
- List of the values or prices of those items, v_i or p_i

Objective: pick some items such as the capacity does not exceed and also the value is maximized.

0/1 Knapsack Problem

Example:

W = 8

w_i = {3, 4, 5, 6}

p_i = {2, 3, 4, 1}

n = 4

Solution 1: Brute Force

0/1 Knapsack Problem

Brute Force Solution:

If we choose a item we put 1 and otherwise 0.

So, the solutions can be

{1, 0, 0, 1}

{0, 1, 1, 0}

.

.

.

.

.

$2^n = 2^4 = 16$ possible solutions

Example:

W = 8

w_i = {3, 4, 5, 6}

p_i = {2, 3, 4, 1}

n = 4

0/1 Knapsack Problem

Algorithm 1 Knapsack(n, W)

```
1: for i ← 0 to n do                          ▷ no remaining capacity
2:     M[i, 0] ← 0
3: end for

4: for wt ← 0 to W do                         ▷ no item to choose from
5:     M[0, wt] ← 0
6: end for

7: for i ← 1 to n do
8:     for wt ← 1 to W do
9:         if w_i > wt then
10:            M[i, wt] ← M[i − 1, wt]
11:        else
12:            M[i, wt] ← max(v_i + M[i − 1, wt − w_i], M[i − 1, wt])
13:        end if
14:    end for
15: end for

16: return M[n, W]
```

Can't take i-th item as its weight W_i exceeds capacity wt

Value after taking the i-th item

Value without taking the i-th item

0/1 Knapsack Problem

Dynamic Programming Solution:

Formula for filling up a cell:

Suppose the cell is M[i, wt]

$$M[i, wt] = \begin{cases} 0 & \text{if } i = 0 \text{ or } wt = 0 \\ M[i-1, wt] & \text{if } w_i > wt \\ \max(v[i] + M[i-1, wt - w[i]], M[i-1, wt]) & \text{otherwise} \end{cases}$$

W = 8

w_i = {3, 4, 5, 6}

p_i = {2, 3, 4, 1}

n = 4

0/1 Knapsack Problem

Fill the tables:

wt

p_i	w_i		0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
2	3	1	0								
3	4	2	0								
4	5	3	0								
1	6	4	0								

i

W = 8

W_i = {3, 4, 5, 6}

P_i = {2, 3, 4, 1}

0/1 Knapsack Problem

Fill the tables:

wt

p_i	w_i		0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
2	3	1	0	0	0	2	2	2	2	2	2
3	4	2	0	0	0	2	3	3	3	5	5
4	5	3	0	0	0	2	3	4	4	5	6
1	6	4	0	0	0	2	3	4	4	5	6

i

Answer

W = 8

W_i = {3, 4, 5, 6}

P_i = {2, 3, 4, 1}

0/1 Knapsack Problem

Find the items:

p_i	w_i	wt	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
2	3	1	0	0	0	2	2	2	2	2	2
3	4	2	0	0	0	2	3	3	3	5	5
4	5	3	0	0	0	2	3	4	4	5	6
1	6 ✗	4	0	0	0	2	3	4	4	5	6

Same, So don't select this

Answer

- Starting from the answer cell, check the upper cell.
- If the values are same, then don't select the item, otherwise select that.
- Now, find out the remaining profit in the upper row.

W = 8

W_i = {3, 4, 5, 6}

P_i = {2, 3, 4, 1}

Solution: { , , ,0}

0/1 Knapsack Problem

Find the items:

p_i	w_i	wt	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
2	3	1	0	0	0	2	2	2	2	2	2
3	4	2	0	0	0	2	3	3	3	5	5
4 ✓	5	3	0	0	0	2	3	4	4	5	6
1	6 ✗	4	0	0	0	2	3	4	4	5	6

Not same, So select this. Remaining Profit: 6 - 4 = 2.

Answer

- Starting from the answer cell, check the upper cell.
- If the values are same, then don't select the item, otherwise select that.
- Now, find out the remaining profit in the upper row.

W = 8

W_i = {3, 4, 5, 6}

P_i = {2, 3, 4, 1}

Solution: { , , 1, 0}

0/1 Knapsack Problem

Now Find the items:

p_i	w_i	wt	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
2	3	1	0	0	0	2	2	2	2	2	2
3	4 ✗	2	0	0	0	2	3	3	3	5	5
4 ✓	5	3	0	0	0	2	3	4	4	5	6
1	6 ✗	4	0	0	0	2	3	4	4	5	6

Same, So don't select this

Answer

- Starting from the answer cell, check the upper cell.
- If the values are same, then don't select the item, otherwise select that.
- Now, find out the remaining profit in the upper row.

W = 8

W_i = {3, 4, 5, 6}

P_i = {2, 3, 4, 1}

Solution: { , 0, 1, 0}

0/1 Knapsack Problem

Now Find the items:

p_i	w_i	wt	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
2 ✓	3	1	0	0	0	2	2	2	2	2	2
3	4	2	0	0	0	2	3	3	3	5	5
4 ✓	5	3	0	0	0	2	3	4	4	5	6
1	6 ✗	4	0	0	0	2	3	4	4	5	6

Answer

Not same, So select this. Remaining Profit: 2-2 = 0. Reached 0th row.

- Starting from the answer cell, check the upper cell.
- If the values are same, then don't select the item, otherwise select that.
- Now, find out the remaining profit in the upper row.

W = 8

W_i = {3, 4, 5, 6}

P_i = {2, 3, 4, 1}

Solution: { 1, 0, 1, 0}